



Personal Wellness & Longevity Coach

A Retrieval-Augmented Generation (RAG) AI Assistant

Course

ITAI 2377 · Data Science

Instructor

Prof. Sitaram Ayyagari

Institution

Houston Community College

Approach

Retrieval-Augmented Generation

Project Team

Juphens Cherfils

Executive Summary & Presentation

Group 1

Kaden Glover

Evaluation Framework & Testing

Group 1

Richard Rodriguez

Domain Research & Project Definition

Group 1

Yoana Cook

Data Architecture, Pipeline & Technical Lead

Group 1

What Is VitalQ?

The problem we solve and how we solve it

The Problem

- Health information online is contradictory & unverified
- 35 million peer-reviewed studies sit in PubMed
- Most people have access to zero of them
- No way to know what is actually backed by science

The Solution — VitalQ

- Ask a health question in plain English
- System searches 520 real PubMed abstracts
- Finds the 5 most relevant research excerpts
- Returns a clear, cited, verifiable answer

PubMed-Grounded

Every answer retrieves from 520 peer-reviewed abstracts via FAISS semantic search

520

PubMed Abstracts

Always Cited

Inline citations [1][2] link directly to PubMed & DOI — every claim is verifiable

1,082

Research Chunks

Consensus Meter

Classifies each source as Supports / Possibly / Mixed / Opposes — shows full picture

53

USDA Foods

RAG Architecture

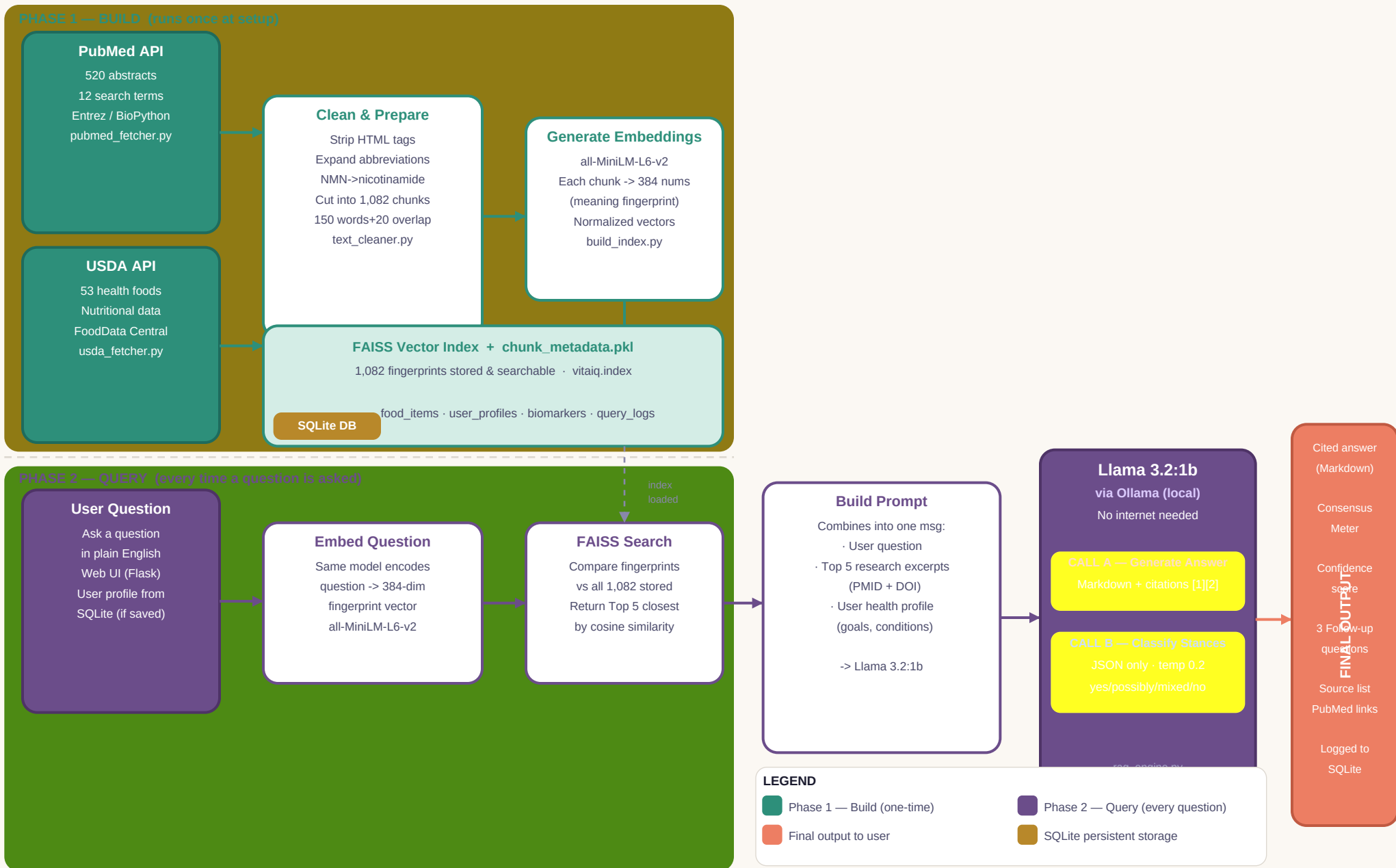
Not hallucinated from memory — grounded in retrieved research documents

100%

Cited Answers

Application Flow Diagram

From data collection to final output



The Metadata Pickle File Explained

What it is, where it comes from, and why it matters

What is `chunk_metadata.pkl`?

FAISS stores only numbers (the fingerprint vectors). It has no idea what study those numbers came from.

The metadata pickle file is the lookup table that maps each vector back to its original PubMed study.

They are created together and always stay in sync — vector #47 in FAISS matches entry #47 in the pickle file.

Pickle is just Python's way of saving a Python list or dictionary to disk so it can be reloaded exactly as it was.

Where it's created

`src/embeddings/build_index.py`

At the end of `build_index.py`, after all 1,082 chunks are embedded, we save the metadata list as a pickle file alongside the FAISS index.

Line: `pickle.dump(metadata, f)`

What it contains

`data/processed/chunk_metadata.pkl`

For each of the 1,082 chunks it stores:

- `doc_id` (unique ID)
- `pmid` (PubMed ID)
- `title` of the study
- `doi` (link to paper)
- `pub_date` (year published)
- `chunk_index` (which piece)
- `chunk_text` (the actual words)

How it's used

`src/retrieval/rag_engine.py`

When FAISS returns 'index 47 is the closest match', we look up position 47 in the metadata list. That's how we know the study title, PMID, and DOI to show the user as a clickable citation link.

Simple analogy

Think of FAISS as a library's card catalog with shelf numbers. The pickle file is the catalog cards themselves — they tell you the book title, author, and location for each shelf number. Without it, you'd find the right shelf but not know what book is there.

Midterm Blueprint vs Final Implementation

What changed, what stayed the same, and why.

Component	Midterm Plan	Final Implementation	Status
Language Model	Anthropic Claude API (pay-per-use)	Llama 3.2:1b via Ollama (free, runs locally)	Changed Better for demo
Data Sources	PubMed + USDA (planned)	PubMed + USDA (fully implemented)	Same ✓
Vector Store	FAISS (planned)	FAISS IndexFlatIP 1,082 vectors, dim=384	Same ✓
Database	SQLite — 5 tables (planned)	SQLite — 5 tables built food_items, profiles, etc.	Same ✓
Text Cleaning	spaCy sentence tokenizer	Regex-based splitter (spaCy incompatible w/ Py3.13)	Adapted
Embedding Model	all-MiniLM-L6-v2 (planned)	all-MiniLM-L6-v2 (implemented exactly)	Same ✓
Web Interface	Flask REST API (planned)	Flask + full UI History, Profile, Biomarkers, Admin	Enhanced
Deployment	Localhost only (planned)	ngrok public tunnel Live demo URL	Enhanced
LLM Calls	1 call per query (planned)	2 calls: answer + JSON classification (Consensus Meter)	Enhanced
Hosting	Local only	ngrok tunnel — public URL for demo day	Enhanced

Key takeaway: The core architecture was planned correctly and implemented as designed.

The main change was the LLM — switching from paid API to free local model — which was a better decision for reliability and cost.

Challenges We Faced

What went wrong, how we solved it, and what we learned

spaCy Incompatibility

Technical

Problem:

spaCy (our sentence tokenizer) failed to install on Python 3.13 + Anaconda — a known broken combination. The blis C library hung indefinitely during

Solution:

Replaced spaCy entirely with a lightweight regex sentence splitter built into Python's standard library. Zero dependencies, same functionality, no installation required.

PyTorch / FAISS Conflict

Technical

Problem:

After installing dependencies via conda, torch 2.4.1 loaded a broken dylib (libtorch_cpu.dylib missing symbols), causing every query to crash with a

Solution:

Uninstalled torch completely — we don't use PyTorch directly. FAISS and sentence-transformers brought in their own compatible versions. Conflict resolved.

RAM / Slow Startup

Performance

Problem:

Python 3.13 on Anaconda took 2-3 minutes to import Flask + sentence-transformers on first startup due to slow .pyc file loading and ~200MB free RAM after

Solution:

Switched to lazy loading (model loads only on first query, not at startup). Stopped postgres background processes. Installed llama3.2:1b (800MB) instead of 3B (2GB). Startup now fast.

pyngrok Hanging

Deployment

Problem:

pyngrok tried to download its own ngrok binary at runtime, conflicting with the system ngrok CLI already installed via Homebrew, causing indefinite

Solution:

Bypassed pyngrok entirely. Used system ngrok CLI directly (ngrok http 5000) which connected instantly using the saved auth token in ~/.Library/Application Support/ngrok/ngrok.yml.

Port 5000 Conflict

Deployment

Problem:

macOS AirPlay Receiver occupies port 5000 by default, preventing Flask from binding and causing 'Address already in use' errors on startup.

Solution:

Disabled AirPlay Receiver in System Settings → AirDrop & Handoff. Flask now binds cleanly to port 5000 with no conflict.

API Key Decision

Architecture

Problem:

Initially planned to use Anthropic Claude API for generation, which required a paid account and internet access — creating cost and reliability

Solution:

Switched to Ollama with Llama 3.2:1b running fully locally. Free forever, works offline, starts in seconds, no API key, no internet dependency during the demo.

Key Lesson

Every major challenge had a simpler solution once we stopped trying to force the original plan. Debugging taught us more about the architecture than building it did.